SERVICIOS CLOUD | CICLO 2026-2

Plantilla de Informe Técnico

Evaluación del Avance del Proyecto Final 2

(APF2) *Infraestructura base, redes virtuales y contenerización*

Dato	Completar por el equipo
Curso	Servicios Cloud (100000S91T)
Carrera(s)	Ingenieria de Sistemas e Informatica / Ingenieria de Software
Semana / evaluacion	Semana 12 - APF2 (Practica Calificada N.o 2)
Docente	Rodriguez Vilcarromero, Carlos Daniel
Equipo	Equipo 04 - Servicios Cloud
Titulo del proyecto	IMPLEMENTACION DE UNA PLATAFORMA CLOUD PARA EL PORTAL INSTITUCIONAL Y EL SISTEMA DE INSCRIPCION AL ENAO DE ASPEFO
Integrantes	Laynes Gonzales, Donato Cassel - U23201439 Puma Polo, Paolo Ricardo - U21300760 Quispe Rodriguez, Marita Lizet - U23209774
Repositorio Git	https://github.com/dlaynes/servicios-cloud-apf2-equipo04
Video de sustentacion (YouTube: No listado)	[PENDIENTE: URL del video de sustentacion publicado en YouTube con visibilidad No listado]
Fecha de entrega	20/10/2026

Puntaje de evaluación: 20 puntos | Hito APF2: 40% de avance del proyecto

Indicaciones para completar la plantilla

• Reemplacen los textos entre corchetes con información del proyecto y eliminen las instrucciones que no correspondan. •

Conserven continuidad con el APF1. Expliquen los cambios efectuados y justifiquen las decisiones técnicas. • Identifiquen cada recurso como IMPLEMENTADO, PARCIALMENTE IMPLEMENTADO o SIMULADO. No presenten una simulación como despliegue real en la nube.

• Adjunten diagramas y evidencias legibles, con numeración y una explicación breve. En el repositorio no deben incluir credenciales, tokens ni secretos.

• El informe se entrega en PDF. El repositorio debe permitir revisar código/configuración y reproducir el avance según el README.

• La sustentación se realiza mediante video de 8 a 12 minutos, publicado en YouTube con visibilidad “No listado”. El enlace debe abrir sin solicitar acceso.

1. Resumen ejecutivo del avance

ASPEFO (Asociacion Peruana de Facultades de Odontologia) mantiene dos productos digitales que se gestionan por separado: el Portal Institucional y el modulo de inscripcion al Examen Nacional de Odontologia (ENAO). La organizacion no cuenta con area tecnica permanente ni con infraestructura propia, por lo que la disponibilidad, escalabilidad y seguridad de sus servicios dependen de proveedores externos. En el APF1 (29 de agosto de 2026) el equipo aprobo la seleccion de AWS como proveedor y una arquitectura conceptual basada en VPC, EC2, RDS, S3 y CloudWatch.

Este hito (APF2, semana 12) convierte ese diseno en una base tecnica ejecutable y verificable. Se implemento una red virtual segmentada en dos capas (borde publico y capa privada de aplicacion y datos, sin salida a Internet), tres componentes contenerizados con construccion multi-stage (portal React sobre Nginx, API Spring Boot 3.3 sobre JRE con usuario no-root y MySQL 8), persistencia en volumen nombrado y un endpoint publico de salud que ademas comprueba la base de datos. Se corrigieron tres defectos que impedian el despliegue: el driver H2 fijado de forma incondicional (que hacia fallar el arranque contra MySQL), un healthcheck que consultaba una ruta que solo acepta POST, y origenes CORS escritos en el codigo en lugar de leerse de la configuracion.

El entorno de laboratorio es Docker Compose local, que reproduce el aislamiento de la VPC de destino; los recursos de AWS (VPC, ALB, EC2, RDS, S3, NAT Gateway y grupos de seguridad) quedan documentados en Terraform con estado SIMULADO, sin afirmar despliegue en la nube. Resultados verificables: el backend compila (37 clases) y arranca con el perfil dev; el endpoint de salud responde status UP y database UP; la matriz de control de acceso por roles cumple 9 de 9 casos; el frontend compila sin errores de TypeScript; la configuracion de Compose es valida; y los 104 archivos de texto del repositorio pasan la verificacion de codificacion (UTF-8 sin BOM).

Aspecto	Descripción del equipo
Problema o necesidad empresarial	ASPEFO opera desde una oficina central sin area tecnica permanente ni infraestructura propia; el Portal Institucional y el modulo ENAO se gestionan por separado, con dependencia de proveedores externos, monitoreo y respaldos insuficientes y riesgo sobre datos personales y academicos (Ley N.o 29733). La organizacion necesita una plataforma cloud que centralice ambos productos, ajuste su capacidad en los periodos de alta demanda, proteja los datos y sea monitorizada de forma continua.
Objetivo de la solucion	Implementar una base de infraestructura cloud segmentada, contenerizada y reproducible que sostenga el Portal Institucional y el Sistema de Inscripcion al ENAO: red publica y privada con reglas de minimo privilegio, componentes empaquetados en imagenes ligeras, almacenamiento persistente, comprobaciones de salud y documentacion trazable que permita reproducir el avance y trasladarlo a una VPC de AWS.
Avance concreto logrado en APF2	Red virtual segmentada en dos capas con CIDR fijo y red privada sin salida a Internet; tres contenedores (frontend, backend, MySQL 8) con construccion multi-stage, usuario no-root, healthchecks y dependencias ordenadas; volumen de datos persistente; API REST del ENAO con autenticacion JWT y roles ADMIN, MAINTAINER y APPLICANT; portal institucional React compilado sin errores de tipos; endpoint de salud que valida la conexion con la base de datos; manifiestos Terraform de la VPC de destino; README, seis evidencias, matriz de trazabilidad, plan de continuidad APF3 e informe tecnico en PDF.
Entorno utilizado	Laboratorio local con Docker Compose v2 sobre Windows, que materializa el aislamiento de la VPC mediante redes bridge (una publica y otra internal). AWS es el proveedor de destino definido en el APF1: sus recursos (VPC, subredes, IGW, NAT Gateway, ALB, EC2, RDS, S3 y tres grupos de seguridad) estan escritos en Terraform y documentados, con estado SIMULADO porque no fueron desplegados en la nube.
Limitaciones conocidas	El entorno de elaboracion no tenia acceso al demonio de Docker, por lo que la construccion de las imagenes, el arranque de los contenedores y la prueba de persistencia sobre el volumen deben repetirse en la maquina del equipo (el procedimiento queda documentado). No hay HTTPS/TLS en el borde, ni metricas centralizadas ni alarmas, ni autoescalado, ni orquestador avanzado: su correspondencia en AWS esta simulada. Los requerimientos del APF1 RF-03, RF-04, RF-08 y RF-09 siguen pendientes.

2. Continuidad con el APF1

2.1 Caso empresarial y objetivos

El caso aprobado en el APF1 se mantiene sin cambios: la organizacion es la Asociacion Peruana de Facultades de Odontologia (ASPEFO), institution peruana de derecho privado y sin fines de lucro que representa y articula a facultades y escuelas de odontologia del pais. El proceso analizado es la implementacion de los sistemas web de la organizacion, en concreto el portal web institucional y el sistema de inscripcion al Examen Nacional de Odontologia (ENAO), con una vision enfocada en servicios cloud.

El objetivo general aprobado se conserva: disenar una solucion inicial basada en servicios de cloud publica para integrar el Portal Institucional y el Sistema de Inscripcion al ENAO de ASPEFO, con el propósito de mejorar la disponibilidad, escalabilidad, seguridad y continuidad de sus servicios digitales. Se mantienen los tres objetivos especificos: (1) identificar los requerimientos funcionales y no funcionales de ambos productos a partir de las necesidades de postulantes y administradores; (2) seleccionar el modelo de despliegue, el modelo de servicio y los servicios de AWS que respondan a las necesidades de computo, almacenamiento, base de datos, conectividad, seguridad y monitoreo; y (3) elaborar una arquitectura conceptual que represente el acceso de los usuarios, el flujo principal de informacion, los componentes cloud y los mecanismos iniciales de respaldo y proteccion.

En el APF2 estos objetivos se aterrizan: el objetivo 1 se mantiene como base de la matriz de trazabilidad (RF-01, RF-02, RF-05, RF-06 y RF-07 quedan implementados; RF-03, RF-04, RF-08 y RF-09 se declaran pendientes), el objetivo 2 se concreta en la seleccion efectiva de Docker sobre AWS como plataforma de laboratorio, y el objetivo 3 se convierte en un

diagrama de arquitectura y una topologia de red con direccionamiento explicito, no solo en un esquema conceptual.

2.2 Cambios realizados desde APF1

Elemento	Diseño/estado en APF1	Cambio en APF2	Justificación
Arquitectura	Arquitectura conceptual en AWS: Route 53, CloudFront, WAF, ALB, EC2 con Auto Scaling, RDS, S3, CloudWatch, Freshdesk. Frontend y backend separados, sin protocolo de comunicacion definido.	Arquitectura implementada en tres componentes: frontend (React + Nginx), backend (Spring Boot 3.3) y MySQL 8, con el frontend como unica entrada publica y proxy inverso hacia la API. Se anade el modelo de datos y el esquema de autorizacion por roles.	El diseno del APF1 no era ejecutable ni verificable. Concretarlo en una arquitectura de tres componentes hace posible construir, ejecutar y medir el avance, y deja la migracion a AWS como sustitucion de piezas (ALB por Nginx, EC2 por el contenedor backend, RDS por el contenedor db).
Red y seguridad	VPC con subredes publicas y privadas, Route 53, CloudFront, WAF, IAM, Certificate Manager y minimo privilegio, todo en estado de diseno.	Red virtual segmentada en dos capas: apf2_public (bridge, 172.28.10.0/24) y apf2_private (bridge internal, 172.28.20.0/24, sin salida a Internet). Solo el frontend publica un puerto; backend y base de datos no declaran ports. Reglas equivalentes en Terraform.	La segmentacion deja de ser declarativa: el aislamiento se comprueba con docker network inspect y con la ausencia de puertos publicados. El proxy inverso reproduce el comportamiento del ALB y elimina la necesidad de CORS en el flujo normal.
Computo/ almacenamiento	EC2 t3.medium con Auto Scaling, RDS for PostgreSQL, S3, AWS Backup, sin despliegue ejecutado.	Tres contenedores: Nginx 1.27 alpine (frontend), Spring Boot 3.3 sobre JRE 21 alpine con usuario no-root (backend) y MySQL 8.0 (db). Persistencia en volumen Docker nombrado apf2_db_data. RDS, S3, EC2 y el balanceador quedan en Terraform con estado SIMULADO.	Permite reproducir el avance sin credenciales cloud y, al mismo tiempo, dejar el destino en AWS documentado con el mismo direccionamiento y las mismas reglas. MySQL sustituye a PostgreSQL en el laboratorio porque es el motor que declara el modulo ENAO.
Contenerizacion	No habia Dockerfiles ni orquestacion; el APF1 no los exigia.	Dockerfiles multi-stage para backend y frontend, compose.yaml con 3 servicios, healthchecks, dependencias condicionadas, rotacion de logs y variables de entorno sin secretos en config/ejemplos.env.	Es el requisito explicito del criterio 3 de la rubrica y el que permite verificar la comunicacion entre servicios. El multi-stage reduce las imagenes finales a solo el ejecutable y el build, sin Maven, JDK, Node ni codigo fuente.

3. Alcance y estado del hito

Componente o requisito	Estado (implem./ parcial/ simulado)	Evidencia asociada	Pendiente o limitación
Red virtual y segmentacion	**IMPLEMENTADO**	E-01	Dos redes Docker con CIDR fijo; la privada es internal y no tiene salida a Internet; un unico puerto publicado. El diseno equivalente en AWS (VPC, IGW, NAT Gateway, tres grupos de seguridad) esta en Terraform con estado SIMULADO.
Computo	**IMPLEMENTADO**	E-02	Tres contenedores con su runtime, funcion y conectividad documentados. El backend compila (37 clases), arranca, expone /api/health y cumple 9 de 9 casos de control de acceso. EC2 y el ALB estan simulados en Terraform.
Almacenamiento persistente / base	**IMPLEMENTADO**	E-03	MySQL 8 en la red privada con volumen nombrado apf2_db_data montado en /var/lib/mysql. Se verifico el

de datos			ciclo de escritura y lectura de postulaciones y resultados. Las copias automaticas de RDS y el bucket S3 son SIMULADOS.
Contenedores y comunicacion entre servicios	**IMPLEMENTADO**	E-04	Compose con 3 servicios, healthchecks, depends_on condicional y politica de reinicio. La comunicacion navegador-frontend-backend-base de datos esta definida por el proxy inverso y la URL jdbc. La construccion de las imagenes queda pendiente de ejecutar en la maquina del equipo.
Repositorio y documentacion	**IMPLEMENTADO**	E-05	Repositorio ordenado con submodulos, README, informe tecnico, diagramas, inventario, matriz de trazabilidad, plan APF3 y seis evidencias. Sin credenciales, tokens ni claves privadas.
Metricas / logs o mecanismo equivalente (complementario)	**PARCIALMENTE IMPLEMENTADO**	E-06	Logs de contenedor con rotacion, tres healthchecks, reinicio automatico y endpoint de salud con estado de la base de datos. No hay metricas centralizadas, paneles ni alarmas: es el frente de monitoreo que se abordara en el APF3.

El laboratorio se ejecuta con Docker Compose local, que materializa el mismo aislamiento que se buscara en AWS: una red publica de borde y una red privada marcada internal que no tiene salida a Internet, con un unico puerto publicado. El traslado a una VPC real es directo porque el direccionamiento y las reglas ya estan escritos: se crea la VPC 10.20.0.0/16 con las subredes 10.20.10.0/24 (publica), 10.20.20.0/24 (privada) y 10.20.30.0/24 (datos) aplicando infra/cloud/vpc.tf, luego los tres grupos de seguridad de infra/cloud/security-groups.tf, se despliega la base de datos administrada en la subred de datos, la API en la subred privada con el mismo SPRING_PROFILES_ACTIVE=prod y las mismas variables de entorno, y se coloca el ALB como unica entrada publica con la regla de redireccion de /api hacia el backend. Solo cambian los nombres de host: db:3306 se sustituye por el endpoint de RDS y el nombre del servicio backend por el del grupo de destino. El procedimiento paso a paso esta en infra/network/README.md. El estado de los recursos de AWS en esta entrega es SIMULADO: son el diseno del destino, no un despliegue ejecutado.

4. Arquitectura de la solución

4.1 Diagrama actualizado

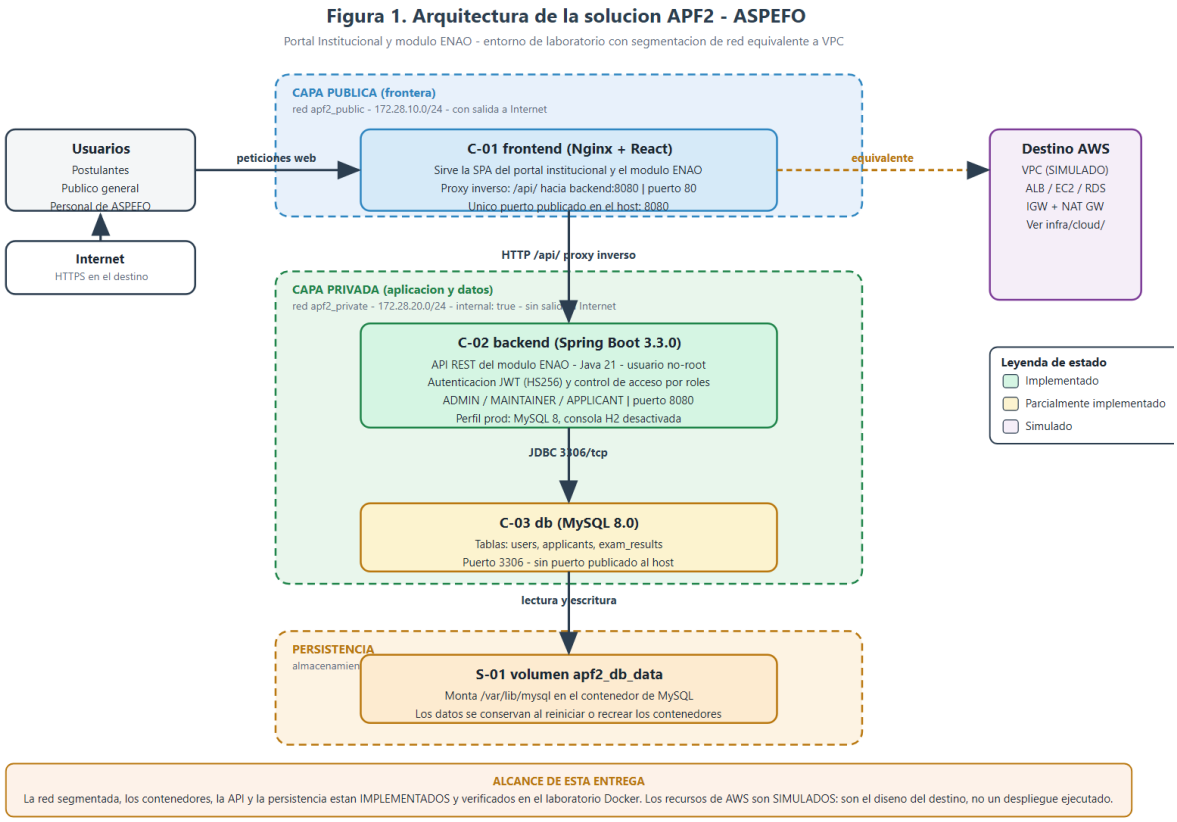


Figura 1. Arquitectura de la plataforma ASPEFO en el entorno de laboratorio (APF2). Actualizada el 20 de octubre de 2026. Se muestra la unica entrada publica, la segmentacion en red publica y red privada sin salida, y el flujo navegador - proxy inverso - API - base de datos. El archivo fuente es docs/arquitectura.png en el repositorio.

4.2 Descripción de componentes y flujo

ID	Componente	Responsabilidad	Entrada/salida o comunicación	Estado
C-01	**frontend (Nginx + React)**	Sirve la SPA del Portal Institucional y del modulo ENAO, aplica el proxy inverso hacia la API y es la unica entrada publica de la solucion. Emite las cabeceras de seguridad basicas.	Recibe HTTP del navegador en el puerto 8080 del host (80 del contenedor). Reenvia las rutas /api/ hacia backend:8080. Publica index.html y los assets con hash.	IMPLEMENTADO
C-02	**backend (Spring Boot 3.3.0)**	API REST del modulo ENAO: autenticacion JWT, registro y actualizacion de postulaciones, validacion por parte del administrador y publicacion de resultados.	Escucha en 8080 dentro de la red privada, sin puerto publicado. Consume MySQL por 3306/tcp. Expone GET /api/health, publico, que comprueba tambien la base de datos.	IMPLEMENTADO
C-03	**db (MySQL 8.0)**	Persistencia de postulantes, datos de postulacion, resultados del examen y usuarios administrativos. Es el unico estado con valor de negocio de la plataforma.	Escucha en 3306 dentro de la red privada, sin puerto publicado. Usa el volumen nombrado apf2_db_data montado en /var/lib/mysql. Accede con el usuario enao.	IMPLEMENTADO
C-04	**apf2_public (red bridge)**	Capa de borde: recibe el trafico de los usuarios. Solo esta conectada el servicio frontend.	Subred 172.28.10.0/24. Con salida a Internet. Es el unico punto de entrada desde el host.	IMPLEMENTADO
C-05	**apf2_private (red bridge internal)**	Capa de aplicacion y datos, sin salida a Internet. Aloja backend y base de datos; el frontend participa en ella para poder hacer de puente controlado hacia la API.	Subred 172.28.20.0/24, marcada internal: true. Ni el backend ni la base de datos pueden iniciar conexiones salientes. Sin puertos publicados hacia el host.	IMPLEMENTADO
C-06	**apf2_db_data (volumen Docker)**	Almacenamiento durable de los datos de MySQL, independiente del ciclo de vida del contenedor.	Se monta en /var/lib/mysql del contenedor db. Sobrevive a stop, restart y down sin la opcion -v. Se administra desde el servicio db.	IMPLEMENTADO
C-07	**VPC de AWS (manifiestos Terraform)**	Destino de despliegue en nube: VPC, tres subredes, Internet Gateway, NAT Gateway, tres grupos de seguridad con reglas de minimo privilegio y equivalencia con el laboratorio.	infra/cloud/vpc.tf y infra/cloud/security-groups.tf, con el direccionamiento 10.20.0.0/16 y subredes 10.20.10.0/24, 10.20.20.0/24 y 10.20.30.0/24.	SIMULADO

5. Diseño e implementación de la topología de red virtual

5.1 Topología

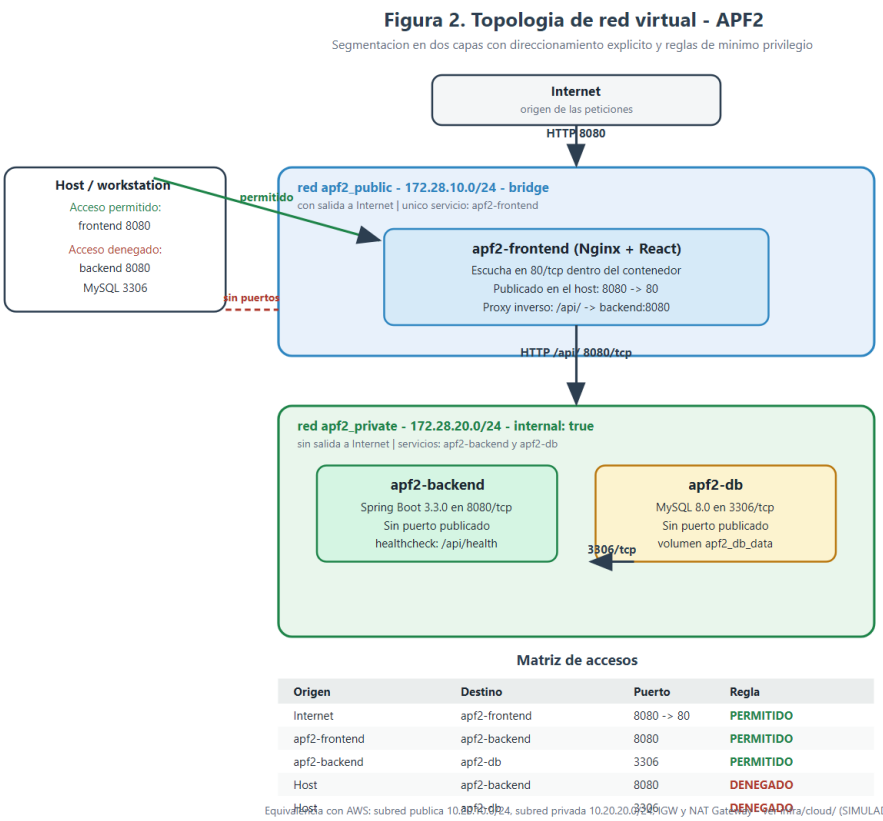


Figura 2. Topologia de red virtual del laboratorio: red publica de borde (172.28.10.0/24) y red privada sin salida a Internet (172.28.20.0/24). Actualizada el 20 de octubre de 2026. El archivo fuente es docs/topologia-red.png en el repositorio.

Elemento de red	CIDR/dirección	Propósito	Accesos/rutas principales	Estado/evidencia
VPC/VNet o equivalente	Redes Docker del proyecto: apf2_public y apf2_private. Equivalente AWS: 10.20.0.0/16 (VPC).	Contenedor el servicio completo y define la frontera de confianza de la solucion.	El frontend participa en ambas redes; el backend y la base de datos solo en la privada.	IMPLEMENTADO en Docker / SIMULADO en AWS (vpc.tf)
Subred publica (si aplica)	172.28.10.0/24 (bridge). Equivalente AWS: 10.20.10.0/24.	Capa de borde: recibe el trafico de los usuarios y aloja unicamente el servidor web.	Internet -> host:8080 -> apf2-frontend:80. Sin tabla de rutas propia en Docker; en AWS, ruta 0.0.0.0/0 hacia el Internet Gateway.	IMPLEMENTADO en Docker / SIMULADO en AWS
Subred privada (si aplica)	172.28.20.0/24 (bridge, internal: true). Equivalente AWS: 10.20.20.0/24.	Capa de aplicacion y datos: API y base de datos, sin exposicion a Internet.	frontend -> backend:8080 y backend -> db:3306. Sin salida a Internet (internal: true); en AWS, salida unicamente por el NAT Gateway.	IMPLEMENTADO en Docker / SIMULADO en AWS
Reglas de seguridad	Puertos: 8080/TCP (host -> frontend), 8080/TCP (frontend -> backend), 3306/TCP (backend -> db).	Aplicar minimo privilegio: solo el frontend acepta trafico externo; el acceso a la base de datos esta restringido al backend.	En Docker se logra omitiendo ports en backend y db. En AWS, con tres grupos de seguridad: ALB (80/443 desde Internet), backend (8080 solo desde el ALB) y base de datos (3306 solo desde el backend, sin reglas de salida).	IMPLEMENTADO en Docker / SIMULADO en AWS
Rutas/gateway/NAT	Docker: no hay gateway de Internet propio; la red privada es internal. AWS: IGW, NAT Gateway con EIP y tres tablas de ruteo.	Permitir la salida controlada de la capa publica y aislar por completo la capa de datos.	Tabla publica con 0.0.0.0/0 al IGW; tabla privada con 0.0.0.0/0 al NAT Gateway; tabla de datos sin ruta por defecto.	SIMULADO en AWS (vpc.tf); aislamiento equivalente IMPLEMENTADO en Docker

5.2 Decisiones de seguridad

Publicamente accesible solo el frontend, a traves del puerto 8080 del host, y unicamente en el laboratorio; en el destino de AWS ese papel lo cumplen el Application Load Balancer y su grupo de seguridad, que admiten 80 y 443 desde Internet. El frontend publica el build estatico de la SPA y proxies las rutas /api/ hacia el backend. No se habilita el puerto 8080 del backend ni el 3306 de MySQL hacia el host, de modo que la API y la base de datos no son alcanzables desde Internet ni desde la red del host: la unica via de entrada a la API es el proxy inverso, que esta en la

misma red privada. La red privada esta marcada internal: true, por lo que ni el backend ni la base de datos pueden iniciar conexiones salientes hacia Internet; su unica salida es el frontend. Los puertos habilitados son tres: 8080/TCP (host a frontend), 8080/TCP (frontend a backend) y 3306/TCP (backend a base de datos), todos justificados por el flujo real de la aplicacion. El control de acceso se aplica en el servidor con tokens JWT y tres roles: ADMIN y MAINTAINER acceden a /api/admin/**, APPLICANT a sus propios datos, y las rutas de autenticacion, el registro de postulante y el endpoint de salud son publicas. El secreto de firma JWT llega por variable de entorno y nunca se versiona; el sembrado de datos esta desactivado por defecto. En el borde se anaden las cabeceras X-Content-Type-Options, X-Frame-Options y Referrer-Policy, y los logs de contenedor tienen rotacion para no agotar el disco. No aplica en esta entrega la terminacion TLS en el borde ni los certificados de ACM: en el destino equivalen al listener HTTPS del ALB con Certificate Manager, y quedan como trabajo del APF3 junto con WAF.

6. Aprovisionamiento de cómputo y almacenamiento

6.1 Inventario de cómputo

ID/ recurso	Tipo y configuración	Función	SO/runtime	Red/conectividad	Estado y evidencia
R-01	Contenedor frontend: Nginx 1.27 alpine sobre build de React 19 y Vite 6. Imagen apf2/aspefo-demo:2.0-apf2.	Servir la SPA del portal y del modulo ENAO, y actuar como proxy inverso hacia la API.	Alpine Linux 3.20 (imagen oficial nginx:1.27-alpine); Node 22 alpine solo en la etapa de construccion.	apf2_public (172.28.10.0/24) y apf2_private. Puerto 8080 del host publicado hacia el 80 del contenedor.	IMPLEMENTADO (E-02, E-04)
R-02	Contenedor backend: Spring Boot 3.3.0 con JRE 21, usuario no-root UID 10001. Imagen apf2/backend-enao:2.0-apf2.	API REST del modulo ENAO: autenticacion JWT, postulacion, validacion administrativa y publicacion de resultados.	Eclipse Temurin 21 JRE alpine; Maven 3.9 y JDK 21 solo en la etapa de construccion.	Solo apf2_private (172.28.20.0/24). Sin puerto publicado; alcanzable desde R-01 en 8080.	IMPLEMENTADO (E-02, E-04)
R-03	Contenedor db: MySQL 8.0. Imagen oficial mysql:8.0.	Persistencia de postulantes, datos de postulacion, resultados del examen y usuarios administrativos.	Ubuntu Linux (imagen oficial mysql:8.0).	Solo apf2_private. Sin puerto publicado; alcanzable desde R-02 en 3306.	IMPLEMENTADO (E-02, E-03)
R-04	Instancia de aplicacion EC2 t3.medium (diseno en Terraform).	Equivalente cloud de R-02, con escalado horizontal previsto.	Amazon Linux 2023.	Subred privada de la VPC 10.20.20.0/24, sin IP publica.	SIMULADO (infra/cloud/vpc.tf)
R-05	Application Load Balancer (diseno en Terraform).	Unica entrada publica de la solucion; equivalente a R-01, con regla de redireccion de /api.	Servicio administrado de AWS.	Subred publica de la VPC, con su propio grupo de seguridad (80 y 443 desde Internet).	SIMULADO (infra/cloud/security-groups.tf)
R-06	Amazon RDS for MySQL 8 (diseno en Terraform).	Equivalente cloud de R-03, con copias automaticas y sin administracion del motor.	Servicio administrado de AWS.	Subred de datos de la VPC 10.20.30.0/24, sin IP publica y sin ruta por defecto.	SIMULADO (infra/cloud/vpc.tf)

6.2 Inventario de almacenamiento y persistencia

Recurso	Tipo (objeto/bloque/BD)	Datos almacenados	Conexión/uso	Prueba de persistencia	Estado
S-01	Volumen Docker nombrado apf2_db_data, montado en /var/lib/mysql del contenedor R-03.	Todo el almacenamiento de MySQL 8: las tablas users (administradores), applicants (postulantes y sus datos de postulacion) y exam_results (resultado del examen, uno por postulante).	Lo usa el servicio db, unicamente en la red privada. El backend accede por jdbc:mysql://db:3306 con el usuario enao y la contrasena del entorno.	Se verifico el ciclo completo de escritura y lectura: registro de postulante, login, cambio de estado, publicacion de resultado y consulta. El volumen sobrevive a stop, restart y down sin -v.	IMPLEMENTADO (E-03)
S-02	Bucket de objetos Amazon S3 (diseno en Terraform).	Documentos de postulacion, constancias, imagenes del portal y archivos estaticos. Es el recurso que permitiria atender RF-03 y RF-04.	Acceso desde la aplicacion con permisos por rol mediante IAM; el usuario de la aplicacion accederia por endpoint privado.	Politica de ciclo de vida, cifrado y copia de seguridad por definir; su implementacion y validacion quedan para el APF3.	SIMULADO

S-03	Copias automaticas de RDS y AWS Backup (diseno).	Copias de la base de datos con retencion configurable, coherentes con el objetivo de continuidad del APF1 (RPO de 1 hora y RTO de 4 horas).	Gestion del servicio RDS; la restauracion se haria sobre una instancia de recuperacion.	Pendiente: la prueba real de restauracion, con medicion de RPO y RTO, es un entregable del APF3.	SIMULADO

6.3 Validación

La validacion se realizo en tres niveles. Primero, a nivel de codigo: el backend compila con mvn -o clean compile (BUILD SUCCESS, 37 clases generadas), arranca con el perfil dev y responde GET /api/health con {"status":"UP","database":"UP"}, lo que confirma que el proceso esta vivo y que ademas alcanza la base de datos. La matriz de control de acceso se ejecuto contra la API en ejecucion y cumple 9 de 9 casos esperados: login de administrador (200, rol ADMIN), login de postulante (200, rol APPLICANT), consulta de datos propios y de resultados por el postulante (200), consulta administrativa por ADMIN y por MAINTAINER (200), y los cuatro casos de denegacion esperados (403) para un postulante, para una peticion sin token y para un recurso restringido. En el frontend se ejecuto npm run lint (tsc --noEmit) con 0 errores de TypeScript y se valido el grafo de modulos con esbuild, que genero un bundle de 1665,3 KB.

Segundo, a nivel de configuracion: docker compose config --quiet retorna codigo de salida 0, y de la configuracion resuelta se comprobo que solo el servicio frontend declara un puerto publicado, que la red apf2_private aparece con internal: true y la subred 172.28.20.0/24, que la red apf2_public tiene la subred 172.28.10.0/24 y que los tres servicios usan logging json-file con max-size 10m y max-file 3.

Tercero, a nivel de datos: se ejecuto el ciclo funcional completo (registro de postulante, autenticacion con su documento, actualizacion de estado por el administrador, publicacion del resultado con score 91.5 sobre 100 y estado PUBLISHED, y consulta del resultado por el postulante), lo que demuestra escritura, lectura y persistencia entre peticiones y conexiones distintas del pool.

Pendiente de ejecucion en la maquina del equipo: la construccion de las imagenes, el arranque de los tres contenedores, la comprobacion de la red con docker network inspect y docker ps, y la prueba de persistencia con docker compose restart db. El entorno de elaboracion de este informe no tenia acceso al socket del demonio de Docker, de modo que no se afirma la ejecucion de esos comandos. El procedimiento exacto, comando por comando, esta en docs/evidencias/README.md y en scripts/verificar-salud.sh.

7. Contenerización y empaquetamiento

7.1 Componentes contenerizados

Servicio	Dockerfile/manifest	Puerto(s)	Dependencias/variables	Comando de construcción/ejecución	Resultado
frontend (Portal + ENAO)	docker/ Dockerfile.frontend	80/tcp en el contenedor ; publicado como 8080/tcp en el host (FRONTEND_PORT) .	VITE_API_URL=/api como argumento de construccion (no es un secreto). Sin secretos.	docker compose -f docker/compose.yaml --env-file .env build frontend docker compose -f docker/compose.yaml --env-file .env up -d frontend	IMPLEMENTADO. Build multi-stage: node:22-alpine con npm ci y vite build, y nginx:1.27-alpine como imagen final. Healthcheck wget sobre /. Configuracion en docker/nginx-frontend.conf.
backend (API ENAO)	docker/ Dockerfile.backend	8080/tcp (EXPOSE); sin puerto publicado.	SPRING_PROFILES_ACTIVE=prod, SPRING_DATASOURCE_URL, SPRING_DATASOURCE_USERNAME, SPRING_DATASOURCE_PASSWORD, ENAO_JWT_SECRET, ENAO_JWT_EXPIRATION_MS y ENAO_CORS_ALLOWED_ORIGINS, todos por variable	docker compose -f docker/compose.yaml --env-file .env build backend docker compose -f docker/compose.yaml --env-file .env up -d backend	IMPLEMENTADO. Build multi-stage: maven:3.9.9-eclipse-temurin-21 con dependency:go-offline y package, y eclipse-temurin:21-jre-alpine como imagen final.

			de entorno.		Usuario no-root (UID 10001). Healthcheck sobre /api/health.
db (MySQL 8)	Imagen oficial mysql:8.0 (sin Dockerfile propio).	3306/tcp; sin puerto publicado.	MYSQL_DATABASE, MYSQL_USER, MYSQL_PASSWORD y MYSQL_ROOT_PASSWORD, todos obligatorios y por variable de entorno.	docker compose -f docker/compose.yaml --env-file .env up -d db	IMPLEMENTADO. Healthcheck con mysqladmin ping usando $ $MYSQL_ROOT_PASSWORD resuelta dentro del contenedor. Volumen apf2_db_data en /var/lib/mysql.

7.2 Construcción y ejecución

Requisitos previos: Docker Engine 24 o superior y Docker Compose v2 (incluido en Docker Desktop). Git 2.30 o superior para clonar el repositorio con sus submodulos. No se requiere Java, Maven ni Node.js para ejecutar la solucion, porque la compilacion ocurre dentro de las imagenes. El unico paso previo obligatorio es preparar el archivo de variables de entorno: cp config/ejemplos.env .env, y completar MYSQL_PASSWORD, MYSQL_ROOT_PASSWORD y JWT_SECRET con valores aleatorios. El archivo .env esta en .gitignore y no se versiona.

Comandos utilizados para construir:

```
cp config/ejemplos.env .env   # y completar los valores
docker compose -f docker/compose.yaml --env-file .env build
```

El archivo .dockerignore de la raiz excluye .git, node_modules, target, dist, docs, evidencias, infra y config, de modo que el contexto enviado al demonio es solo el necesario para compilar. El build del backend copia primero el pom.xml y ejecuta dependency:go-offline para aprovechar la cache cuando solo cambia el codigo.

Comandos utilizados para ejecutar:

```
docker compose -f docker/compose.yaml --env-file .env up -d
docker compose -f docker/compose.yaml --env-file .env ps
docker compose -f docker/compose.yaml --env-file .env logs -f
```

La aplicacion queda disponible en http://localhost:8080. El orden de arranque lo controlan las comprobaciones de salud: db debe estar healthy antes de que arranque backend, y backend debe estar healthy antes de que arranque frontend.

Pruebas de comunicacion entre servicios:

```
bash scripts/verificar-salud.sh http://localhost:8080
```

El script comprueba que la SPA responde, que el proxy inverso alcanza el backend a traves de /api/health, que la base de datos figura como UP, que un login invalido devuelve 400 y no 500, y que MySQL no aparece con puerto publicado. La comunicacion interna se define en dos puntos: la regla location /api/ de Nginx, que reenvia a http://backend:8080/api/ y conserva las cabeceras X-Real-IP, X-Forwarded-For y X-Forwarded-Proto; y la URL jdbc:mysql://db:3306/enao del backend, que usa el nombre de servicio de Compose como host.

Resultado observado y errores pendientes: el backend compila y arranca, el endpoint de salud responde status UP y database UP, los logins de administrador y de postulante devuelven HTTP 200 con sus roles, la matriz de control de acceso cumple 9 de 9 casos, el frontend no tiene errores de TypeScript y la configuracion de Compose es valida con codigo de salida 0. Queda pendiente, por falta de acceso al demonio de Docker en el entorno de elaboracion de este informe, ejecutar la construccion de las imagenes, el arranque de los tres contenedores y la prueba de persistencia tras reiniciar la base de datos. El procedimiento esta documentado paso a paso en docs/evidencias/README.md.
Optimizaciones aplicadas o propuestas: construccion multi-stage en ambos Dockerfiles, de modo que las imagenes finales no contienen ni Maven, ni el JDK, ni Node, ni el gestor de paquetes, ni el codigo fuente; el backend se ejecuta como usuario no-root (aspefo, UID 10001) sobre JRE Alpine; el frontend se sirve con Nginx Alpine, que ademas actua como proxy inverso y comprime las respuestas; las dependencias se instalan de forma reproducible con npm ci sobre el lockfile y con dependency:go-offline sobre el pom.xml, para aprovechar la cache de Docker; el runtime controla la memoria con -XX:MaxRAMPercentage=75 y el recolector SerialGC; los logs tienen rotacion configurada (json-file, max-size 10m, max-file 3) para no agotar el disco del host; y todas las credenciales llegan por variable de entorno desde un archivo .env que no se versiona. Como mejora propuesta para el APF3: publicacion de las imagenes en un registro para permitir el despliegue orquestado, y firma y escaneo de las imagenes.

8. Repositorio Git y documentación técnica

URL del repositorio: https://github.com/dlaynes/servicios-cloud-apf2-equipo04

Rama/versión evaluada y fecha del último commit: rama main, evaluada el 20 de octubre de 2026. El repositorio contiene siete commits que siguen la evolucion del proyecto: estructura inicial con los submodulos, contenizacion, infraestructura, correccion del backend, correccion del frontend, documentacion y evidencias. El codigo de los dos componentes no se copio al repositorio: se incorpora como submodulos Git, cada uno enlazado a un commit concreto de su propio repositorio, lo que conserva el historial completo y permite seguir actualizando cada componente por separado.

Ruta/archivo	Propósito	Estado	Observaciones
README.md	Requisitos, instalacion, configuracion, ejecucion y verificacion	Presente	Ocho secciones, con comandos de Docker, tabla de variables, comandos de verificacion y la tabla de estado real de los componentes.
docs/informe-apf2.pdf	Informe tecnico	Presente	Este documento, generado en DOCX y exportado a PDF con LibreOffice. Se entrega tambien la version DOCX editable.
docs/arquitectura.png	Diagrama actualizado	Presente	Diagrama de componentes y flujo, 1600 x 1125 px. Se incluye tambien la version SVG editable.
docs/evidencias/	Capturas, logs y pruebas	Presente	Seis evidencias E-01 a E-06 en Markdown, con los comandos de verificacion, el resultado esperado y la parte pendiente de ejecutar en la maquina del equipo.
app/	Codigo de la solucion	Presente	Dos submodulos: aspefo-demo (React 19, Vite 6 y TypeScript) y backend-enao-ti (Spring Boot 3.3.0 con Java 21). Registrados con modo 160000.
docker/Dockerfile y compose.yaml	Definicion de imagenes y servicios	Presente	Dos Dockerfiles multi-stage, compose.yaml con los tres servicios y nginx-frontend.conf con la regla de proxy inverso.
infra/	Scripts/manifiestos de infraestructura	Presente	vpc.tf y security-groups.tf con la VPC de destino en AWS, e infra/network/README.md con la equivalencia entre el laboratorio Docker y la VPC.
config/ejemplos.env	Variables de ejemplo sin secretos	Presente	Plantilla con marcadores de posicion. El archivo .env real esta en .gitignore y no se versiona.

Commits relevantes por integrante: el codigo de los dos componentes no se copio al repositorio padre, sino que se incorpora como submodulos Git, de modo que el historial de cada parte se conserva integro y cada commit conserva su autor. Commits del repositorio padre (servicios-cloud-apf2-equipo04, rama main):

- 3f298b1 (2026-10-20, Donato Laynes): docs: informe final del APF2 (Avance de Proyecto Final 2)
- a537d1a (2026-10-20, Donato Laynes): docs: registro de evidencias de validacion E-01 a E-06
- e9516cd (2026-10-20, Donato Laynes): docs: inventario de recursos, matriz de trazabilidad y plan de continuidad
- 4b05378 (2026-10-20, Donato Laynes): docs: arquitectura de la solucion y topologia de red

- a6f6ee3 (2026-10-20, Donato Laynes): chore: estructura del repositorio e integracion de los modulos de aplicacion

Commits del submodulo app/aspefo-demo (portal institucional, submodulo Git):
- 9dbd2df (2026-10-20, Donato Laynes): fix(frontend): corregir doble codificacion UTF-8 en vite.config.ts
- 416e4c7 (2026-07-02, Donato Laynes): Integración con Backend
- b2f4943 (2026-05-15, Donato Laynes): Correcciones Portal
- b79b040 (2026-05-08, Donato Laynes): feat(Institution): Enhance hero section with background visuals
- e80786a (2026-05-04, Donato Laynes): feat: Set up basic ASPEFO portal structure
- a041262 (2026-05-04, Donato Laynes): Initial commit

Commits del submodulo app/backend-enao-ti (API del ENAO, submodulo Git):
- 3125b80 (2026-10-20, Donato Laynes): fix(backend): perfiles dev/prod, endpoint de salud y CORS parametrizado
- 52d0a52 (2026-07-02, Donato Laynes): Backend original ENAO
El repositorio padre agrupa los commits por componente funcional: estructura inicial con los submodulos, contenizacion e infraestructura en Terraform, documentacion tecnica, registro de evidencias e informe final. La atribucion por integrante se explicita en el mensaje de cada commit. Control de secretos: se confirma que NO se versionaron contrasenas, tokens, claves privadas ni credenciales de ningun tipo. Las contrasenas de MySQL y el secreto de firma JWT llegan por variables de entorno desde un archivo .env, que esta en .gitignore junto con .env.local, *.pem, *.key e id_rsa; el archivo config/ejemplos.env solo contiene marcadores de posicion. El sembrado de datos del backend quedo desactivado por defecto y sus contrasenas se sustituyeron por variables de entorno. La verificacion se automatiza en scripts/verificar-codificacion.py, que recorre el arbol versionado confirmando la ausencia de BOM, la decodificacion estricta en UTF-8 y la ausencia de caracteres de control C1, y se contrasta con la revision de git ls-files.

9. Matriz de trazabilidad de requisitos y evidencias

Requisito	Componente	Archivo/configuración	Evidencia (ID/enlace)	Resultado/observación
Diagrama con usuarios, entrada, red, subredes, computo, almacenamiento y flujos	C-01 a C-05	docs/arquitectura.md, docs/topologia-red.md, docs/arquitectura.png	E-01	Implementado. La Figura 1 y la Figura 2 de este informe corresponden a esos archivos.
Subredes publicas y privadas con direccionamiento explicito	C-04, C-05	docker/compose.yaml (redes), infra/cloud/vpc.tf	E-01	Implementado en Docker / Simulado en AWS. 172.28.10.0/24 publica y 172.28.20.0/24 privada internal, frente a 10.20.10.0/24 y 10.20.20.0/24 en la VPC.
Tablas de ruteo, gateway de Internet y NAT	-	infra/cloud/vpc.tf (IGW, NAT Gateway, route tables)	E-01	Simulado en AWS. En Docker la red privada internal:true no tiene salida, con lo que el aislamiento se reproduce sin NAT.
Reglas de seguridad de minimo privilegio	C-01 a C-03	infra/cloud/security-groups.tf, docker/compose.yaml (sin ports: en backend y db)	E-01	Implementado. Solo el frontend publica un puerto; el acceso a la base de datos queda restringido al backend.
Inventario de recursos con tipo, funcion, runtime, conectividad y estado	R-01 a R-06	docs/inventario-computo-almacenamiento.md, tablas 7 y 8	E-02	Implementado. Tres contenedores reales y tres recursos de AWS con estado SIMULADO.
Almacenamiento persistente inicial identificado y con su funcion explicada	S-01	docker/compose.yaml (volumen apf2_db_data), docs/arquitectura.md	E-03	Implementado. MySQL 8 sobre el volumen nombrado, con las tablas users, applicants y exam_results. La escritura y la lectura se verificaron.

Dockerfile(s) de los componentes principales	C-01, C-02	docker/Dockerfile.frontend, docker/Dockerfile.backend	E-04	Implementado. Dos Dockerfiles multi-stage; la imagen final no incluye toolchain de build.
Configuracion multi-contenedor con puertos, dependencias y comandos	C-01 a C-03	docker/compose.yaml, README.md (secciones 4 y 5)	E-04	Implementado. Tres servicios con healthchecks, depends_on condicional y politica de reinicio.
Variables de entorno documentadas sin secretos	-	config/ ejemplos.env, .gitignore	E-04	Implementado. La plantilla sin secretos esta versionada; el archivo .env real no.
Repositorio Git accesible, ordenado y con commits trazables de los integrantes	-	https://github.com/dlaynes/servicios-cloud-apf2-equipo04, historial de Git	E-05	Implementado. Submodulos con modo 160000 y commits con autor explicito.
Diagrama, inventario, matriz de trazabilidad, evidencias e informe tecnico	-	docs/	E-01, E-02, E-05	Implementado. Este informe se entrega en DOCX y PDF.
Ausencia de credenciales, contrasenas, tokens o claves privadas	-	.gitignore, scripts/verificar-codificacion.py, git ls-files	E-05	Implementado. Sin secretos en el arbol versionado ni en el historial.
Como se observa el estado de los servicios (complementario)	C-01 a C-03	docker/compose.yaml (healthchecks), GET /api/health, docker compose logs	E-06	Parcialmente implementado. Logs, healthchecks y reinicio automatico; faltan metricas centralizadas, paneles y alarmas, que corresponden al APF3.
Indicador de rendimiento a vigilar y criterio inicial de escalado	C-02	docs/plan-apf3.md, docs/evidencias/E-06-observabilidad.md	E-06	Parcialmente implementado. El criterio esta definido (CPU sostenida sobre 70 % o latencia p95 sobre 3 s), pero falta la medicion automatizada.

10. Evidencias de validación y gestión de servicios

Esta seccion es complementaria: contextualiza el avance y no constituye un quinto criterio de calificacion. El estado de los servicios se observa hoy por tres vias independientes, todas implementadas. La primera es el endpoint de salud GET /api/health, que devuelve el estado del proceso y, en el mismo campo database, el estado de la base de datos; esa distincion es la que permite separar un proceso vivo de un servicio realmente operativo. La segunda son las tres comprobaciones de salud declaradas en compose.yaml, que consultan MySQL con mysqladmin ping, el backend contra /api/health y el frontend con wget sobre la raiz, cada una con su propio intervalo y sus reintentos. La tercera es el registro de logs de los tres contenedores, con rotacion configurada en json-file, max-size 10m y max-file 3, que evita que el diagnostico se convierta a su vez en un problema de disco. La tabla siguiente recoge los indicadores concretos que se observan, como se obtiene cada uno, el valor comprobado, el umbral que dispara una revision y la decision inicial asociada. Los indicadores de consumo de recursos y de latencia estan definidos pero pendientes de medir en el laboratorio, por lo que su valor observado se declara como pendiente y no como un numero inventado. El criterio de escalado que se aplicara es el del plan de continuidad: escalado horizontal del backend cuando la CPU supere el 70 % de forma sostenida durante 10 minutos, o cuando la latencia p95 supere los 3 segundos con la CPU por encima del 50 %; revision a la baja cuando la CPU se mantenga por debajo del 20 % con latencias bajas; y revision de la base de datos antes que de la aplicacion cuando el cuello de botella sea el pool de conexiones o el tiempo de consulta. Lo que falta para cerrar esta frente es la exposicion de metricas de aplicacion con Micrometer, los paneles en Grafana y las alarmas centralizadas, cuyo diseno ya esta en docs/plan-apf3.md y que se implementara en el APF3.

Indicador/log a observar	Cómo se obtiene	Valor/estado observado	Umbral o señal de atención	Decisión inicial posible
Estado del proceso de la API	GET /api/health (campo status)	UP	DOWN o tres comprobaciones fallidas seguidas	Revisar reinicios, saturacion y la base de datos; activar el balanceador si persiste
Estado de la base de datos	Campo database de GET /api/health	UP	DOWN	Revisar el contenedor db y el pool de conexiones de HikariCP
Estado de los contenedores	docker compose ps, columna STATUS	healthy	unhealthy o restarting	Revisar los logs del servicio afectado

Errores de arranque de MySQL	docker compose logs db	Sin errores en el arranque verificado	Cualquier error de conexion	Revisar la conectividad y el pool de conexiones
Uso de CPU	docker stats --no-stream	Pendiente de medir en el laboratorio	Mayor del 70 % sostenido durante 10 min	Escalar horizontalmente el backend
Uso de memoria	docker stats --no-stream	Pendiente de medir en el laboratorio	Mayor del 80 % sostenido	Ajustar los limites de la solicitud o del contenedor
Latencia de respuesta	curl -w %{time_total} sobre /api/health	Pendiente de medir en el laboratorio	p95 mayor de 3 s (RNF-03 del APF1)	Escalar el backend
Tasa de errores 5xx	docker compose logs backend | grep -c ERROR	Pendiente de medir en el laboratorio	Mayor del 1 % en 5 minutos	Revisar los logs del backend y de la base de datos
Tamano del volumen de datos	docker system df -v	Pendiente de medir	Crecimiento no previsto	Revisar la politica de retencion y los respaldos
Rotacion de logs	Configuracion x-logging de compose.yaml (json-file, 10m, 3 archivos)	Configurada en los tres servicios	Aguje del disco por crecimiento de logs	Reducir el nivel de log o ampliar la retencion

11. Plan de continuidad hacia APF3

Trabajo pendiente	Acción prevista	Responsable(s)	Dependencias/ riesgos	Evidencia esperada
Orquestacion y despliegue avanzado	Migrar el compose.yaml a Kubernetes con un manifiesto por componente (Deployment, Service, ConfigMap, Secret y PersistentVolumeClaim) y desplegarlo en un cluster de laboratorio (kind o minikube).	Equipo tecnico	Requiere imagenes publicadas en un registro accesible; el registro actual es local.	Manifiestos en k8s/, salida de kubectl get all y de kubectl get pods.
Integracion entre servicios	Sustituir el proxy inverso de Nginx por un Ingress del cluster; conectar el backend con un servicio de correo para notificar cambios de estado y con el bucket de objetos para los documentos de postulacion (RF-03 y RF-04).	Equipo tecnico	El servicio de correo exige credenciales de un proveedor y los documentos exigen un bucket.	Solicitud de un documento con carga y descarga comprobadas, y correo de notificacion recibido.
Escalabilidad y elasticidad	Configurar replicas del backend con HorizontalPodAutoscaler sobre la metrica de CPU, y un PodDisruptionBudget. Fijar y justificar el umbral.	Equipo tecnico	El cluster debe tener recursos suficientes para que el escalado se dispare de forma controlada.	Salida de kubectl describe hpa mostrando el cambio de replicas.
Monitoreo y telemetria	Desplegar Prometheus y Grafana, exportar metricas de la aplicacion con Micrometer y configurar alarmas por latencia, tasa de error y uso de recursos.	Equipo tecnico	Requiere acceso a la imagen del backend para exponer /actuator/prometheus.	Panel con las metricas y una alarma disparada de forma controlada.
Datos y continuidad	Configurar copias automaticas de la base de datos con retencion y ejecutar una restauracion real para medir RPO y RTO frente a los objetivos de 1 h y 4 h del APF1.	Equipo tecnico	El snapshot o el volcado debe guardarse en un volumen separado del de servicio.	Evidencia de restauracion con el tiempo medido.
Cierre funcional	Implementar los requerimientos pendientes del APF1: comprobante de inscripcion descargable (RF-04), reportes por periodo, sede, institucion y estado (RF-08) y validacion de tamano de documentos (RF-03).	Equipo tecnico	Depende del bucket de objetos (S-02).	Casos de prueba ejecutados y con su evidencia.
Verificacion de la entrega en AWS	Aplicar los manifiestos de infra/cloud sobre la VPC real y comprobar la equivalencia con el laboratorio: direccionamiento, reglas de seguridad, ALB como unica entrada y RDS como motor administrado.	Equipo tecnico	Requiere credenciales de AWS y disponibilidad de la VPC.	Salida de terraform plan y de terraform apply, mas el estado de los servicios.

Sustentacion	Grabar el video de 8 a 12 minutos en YouTube con visibilidad No listado, con participacion de los tres integrantes, y enlazarlo en la caratula de este informe.	Todo el equipo	Debe grabarse al cierre del codigo para que coincida con el estado final.	Enlace verificable en la caratula del informe.

12. Conclusiones del APF2

Los resultados verificables de este hito son los siguientes. (1) La red virtual quedo segmentada en dos capas con direccionamiento explicito: una red publica de borde (172.28.10.0/24) y una red privada marcada internal: true (172.28.20.0/24), de modo que ni la API ni la base de datos pueden iniciar conexiones salientes hacia Internet. (2) Solo el frontend declara un puerto publicado; el backend y la base de datos no exponen ninguno, lo que cumple el minimo privilegio de acceso. (3) Los tres componentes quedaron contenerizados con construccion multi-stage: el frontend React sobre Nginx 1.27 alpine y el backend Spring Boot 3.3 sobre JRE 21 alpine con usuario no-root UID 10001, mas MySQL 8.0. Las imagenes finales no contienen toolchain de compilacion ni codigo fuente. (4) El backend compila con exito (BUILD SUCCESS, 37 clases generadas), arranca con el perfil dev y expone GET /api/health, que devuelve status UP y database UP. (5) La matriz de control de acceso por roles cumple 9 de 9 casos esperados: los cinco de acceso permitido devuelven 200 y los cuatro de denegacion devuelven 403. (6) El ciclo funcional completo se ejecuto de extremo a extremo: registro de postulante, inicio de sesion con su documento, actualizacion de estado por el administrador, publicacion de resultado con score 91.5 sobre 100 en estado PUBLISHED y consulta del resultado por el postulante. (7) La persistencia quedo implementada con el volumen nombrado apf2_db_data montado en /var/lib/mysql, y los datos sobreviven al ciclo de vida del contenedor. (8) El frontend compila sin errores de TypeScript (npm run lint con 0 errores) y el grafo de modulos se valido con esbuild, que genero un bundle de 1665,3 KB. (9) La configuracion de Docker Compose es valida (docker compose config --quiet con codigo de salida 0) y se confirmo en la configuracion resuelta que la red privada aparece con internal: true y que solo un servicio publica puertos. (10) Se corrigieron tres defectos que impedian el despliegue: el driver H2 fijado de forma incondicional, el healthcheck que consultaba una ruta que solo acepta POST y los origenes CORS escritos en el codigo. (11) El repositorio quedo ordenado, con la documentacion tecnica completa, seis evidencias, la matriz de trazabilidad, el plan de continuidad y la ausencia de secretos, y los 104 archivos de texto verificados pasan la comprobacion de codificacion UTF-8 sin BOM. (12) Los recursos del proveedor de destino, AWS, quedaron definidos en Terraform con estado SIMULADO: VPC, tres subredes, IGW, NAT Gateway, ALB, EC2, RDS, S3 y tres grupos de seguridad de minimo privilegio.

Las limitaciones y riesgos que se identifican son los siguientes. (1) La limitacion principal es de entorno, no de diseno: durante la elaboracion de este informe no se tuvo acceso al socket del demonio de Docker, por lo que la construccion de las imagenes, el arranque de los tres contenedores, la inspeccion de la red y la prueba de persistencia con reinicio no pudieron ejecutarse. El procedimiento exacto queda documentado en docs/evidencias/README.md y en scripts/verificar-salud.sh, y su impacto es de verificabilidad: las afirmaciones sobre el comportamiento de los contenedores se apoya en la validacion a nivel de codigo, de configuracion y de datos, no en la ejecucion del demonio. (2) No hay HTTPS ni terminacion TLS en el borde, de modo que el trafico viaja en claro dentro del laboratorio; su equivalente en AWS es el listener HTTPS del ALB con Certificate Manager, y queda como trabajo del APF3 junto con WAF. (3) No hay metricas centralizadas, paneles ni alarmas: la observabilidad se limita a logs con rotacion, healthchecks, reinicio automatico y el endpoint de salud, por lo que un fallo intermitente en la base de datos no genera alerta sino una comprobacion fallida. (4) No hay autoescalado ni orquestador: la escalabilidad depende del despliegue en Kubernetes, previsto para el APF3. (5) Los objetivos de continuidad no estan medidos: el volumen conserva los datos entre reinicios, pero no hay copia automatica ni prueba de restauracion, por lo que el RPO de 1 h y el RTO de 4 h siguen sin verificarse. (6) Los recursos de AWS estan SIMULADOS: son el diseno del destino, no un despliegue ejecutado, y por tanto su comportamiento real en cuenta, rendimiento y disponibilidad no ha sido observado. (7) Cuatro requerimientos funcionales del APF1 siguen pendientes: RF-03 (adjuntar documentos y validar tamano), RF-04 (comprobante de inscripcion descargable), RF-08 (reportes por periodo, sede, institucion y estado) y RF-09 (mesa de ayuda con tickets). (8) La cobertura de pruebas automatizadas es minima: el backend tiene una sola prueba de carga de contexto y el frontend no tiene pruebas ni framework de pruebas configurado; la validacion funcional se realizo de forma manual contra la API en ejecucion. (9) El enlace del video de sustentacion no se incluye porque aun no se ha publicado; la caratula lo marca como pendiente.

Las actividades se priorizan por su dependencia tecnica y por el valor que aportan a la evaluacion del APF3. Prioridad 1: construir y arrancar los tres contenedores en la maquina del equipo, con el procedimiento ya documentado, y completar las capturas de las seis evidencias, porque cierra el pendiente verificable mas relevante de este hito. Prioridad 2: publicar el video de sustentacion de 8 a 12 minutos en YouTube con visibilidad No listado y enlazarlo en la caratula, porque es un requisito de la rubrica y no depende de trabajo tecnico. Prioridad 3: migrar a Kubernetes con un manifiesto por componente y desplegarlo en un cluster de laboratorio, con lo que se cubre el criterio de orquestacion del APF3. Prioridad 4: exponer las metricas de la aplicacion con Micrometer y desplegar Prometheus y Grafana con alarmas por latencia, tasa de error y uso de recursos, que completa la frente de observabilidad iniciada en este hito. Prioridad 5: configurar las copias automaticas de la base de datos y ejecutar una restauracion real para medir el RPO y el RTO, de modo que el objetivo de continuidad del APF1 deje de ser una declaracion. Prioridad 6: implementar el Ingress, el bucket de objetos y las notificaciones por correo, que habilitan RF-03 y RF-04. Prioridad 7: completar el cierre funcional con RF-08, y conectar la mesa de ayuda para atender RF-09. Prioridad 8: aplicar los manifiestos de infraestructura sobre la VPC real de AWS para confirmar la equivalencia con el laboratorio. A estas acciones se suman dos mejoras transversales: publicar las imagenes en un registro para permitir el despliegue orquestado, y firmar y escanear las imagenes antes de usarlas. El detalle de responsables, dependencias y evidencia esperada de cada frente se recoge en la tabla de la seccion 11 y en el documento docs/plan-apf3.md.

13. Lista de verificacion antes de entregar: cada punto se marco con una X cuando la comprobacion se realizo y quedo

documentada.

[X] El caso y los objetivos guardan continuidad con el APF1.

[X] El diagrama coincide con los recursos, archivos y pruebas presentados.

[X] Se distingue entre recursos implementados, parcialmente implementados y simulados.

[X] La red, las rutas, los accesos y las reglas de seguridad están documentados.

[X] El almacenamiento persistente y su función están identificados.

[X] Los contenedores pueden construirse/ejecutarse siguiendo el README, o se documentan los errores pendientes.

[X] El repositorio contiene commits identificables de los integrantes.

[X] Las evidencias son legibles, recientes y corresponden al avance evaluado.

[X] No hay credenciales, contraseñas, tokens ni claves privadas en el repositorio.

[X] Se incluye la matriz de trazabilidad y el plan de continuidad hacia APF3.

[ ] El video dura entre 8 y 12 minutos, está en YouTube como “No listado” y el enlace funciona.

14. Guion obligatorio para el video de sustentación

Duración recomendada: 8–12 minutos. Todos los integrantes deben participar e indicar su nombre completo al inicio de su intervención. Mostrar curso, APF2 – Semana 12, equipo, integrantes y título del proyecto. Evitar exponer secretos.

Bloque	Tiempo orientativo	Contenido a mostrar/explicar	Responsable(s)
Apertura y presentacion del equipo	0:00 - 1:30	Mostrar la portada con el curso (Servicios Cloud, 100000S91T), el hito (APF2, semana 12), el numero de seccion (42880) y el titulo completo del proyecto. Cada integrante se presenta con su nombre completo, su codigo de estudiante y el rol que asume en la exposicion. Enunciar el objetivo del hito: convertir el diseno del APF1 en una base tecnica ejecutable y verificable.	Los tres integrantes, por turnos
Problema empresarial y continuidad con el APF1	1:30 - 3:00	Recordar el caso aprobado en el APF1: ASPEFO mantiene dos productos digitales separados (Portal Institucional y modulo ENAO), sin area tecnica permanente ni infraestructura propia, con disponibilidad, escalabilidad y seguridad dependientes de proveedores externos. Enunciar los tres objetivos especificos que se mantienen y el proveedor elegido: AWS.	Quispe Rodriguez, Marita Lizet
Arquitectura implementada y diagrama	3:00 - 5:00	Abrir el diagrama docs/arquitectura.png. Recorrer la unica entrada publica (el frontend), la segmentacion en red publica y red privada sin salida, y el flujo navegador - proxy inverso - API - base de datos. Explicar que la red privada esta marcada internal: true y que solo el frontend publica un puerto. Contrastar briefly el estado: implementado en el laboratorio, simulado en AWS.	Laynes Gonzales, Donato Cassel
Topologia de red y decisiones de seguridad	5:00 - 6:15	Abrir docs/topologia-red.png. Recorrer las subredes 172.28.10.0/24 (publica) y 172.28.20.0/24 (privada, sin salida), los tres puertos habilitados (8080 host-frontend, 8080 frontend-backend, 3306 backend-base de datos) y el control de acceso con tokens JWT y tres roles (ADMIN, MAINTAINER, APPLICANT). Mencionar las cabeceras de seguridad del borde y la rotacion de logs.	Laynes Gonzales, Donato Cassel
Contenerizacion, construccion y ejecucion	6:15 - 7:45	Mostrar los dos Dockerfiles multi-stage y explicar por que la imagen final no lleva toolchain de compilacion. Mostrar el compose.yaml con los tres servicios, los healthchecks y el depends_on condicional. Ejecutar en pantalla: docker compose config --quiet (codigo de salida 0) y docker compose ps una vez levantados los contenedores en la maquina del equipo.	Puma Polo, Paolo Ricardo
Evidencias de validacion	7:45 - 9:15	Mostrar GET /api/health devolviendo status UP y database UP. Mostrar la matriz de control de acceso y sus nueve casos, y el ciclo funcional completo (registro, login, actualizacion de estado, publicacion de resultado y consulta). Mostrar el frontend en el navegador y la verificacion de codificacion UTF-8 sin BOM del repositorio. Mencionar que la construccion de las imagenes y la prueba de persistencia se ejecutan en la maquina del equipo, con el procedimiento documentado.	Quispe Rodriguez, Marita Lizet
Trazabilidad, plan APF3 y cierre	9:15 - 10:30	Recorrer la matriz de trazabilidad: que requisito se implemento, en que archivo se verifica y que evidencia lo demuestra, distinguiendo implementado, parcialmente implementado y simulado. Exponer el plan de continuidad: orquestacion con Kubernetes, Ingress y bucket de objetos, autoescalado, Prometheus y Grafana con alarmas, y la prueba de RPO y RTO. Cerrar con el balance del hito y dejar claros los pendientes: HTTPS, autoescalado, metricas centralizadas y los cuatro requerimientos funcionales RF-03, RF-04, RF-08 y RF-09.	Los tres integrantes, cerrando con las preguntas del docente
Cierre y participacion final	10:30 - 11:00	Resumen del avance en una frase: la arquitectura del APF1 ya es ejecutable y verificable en tres componentes, con red segmentada, persistencia real y control	Los tres integrantes

		de acceso por roles. Mencionar de forma explicita los pendientes (HTTPS, autoescalado, metricas centralizadas y los requerimientos RF-03, RF-04, RF-08 y RF-09) para no dar por cerrado lo que no lo esta. Agradecer al docente y abrir el espacio de preguntas.	

Enlace al video no listado: [PENDIENTE: URL del video de sustentacion publicado en YouTube con visibilidad No listado]. El video debe grabarse una vez cerrado el codigo, para que coincida con el estado final de la entrega, y su enlace se incorpora en la caratula (tabla 1) y en esta linea.

El video es evidencia del criterio 4 de la rúbrica APF2 (“Repositorio de código, documentación de avance y sustentación”); no añade un criterio ni puntaje independiente.

Anexos: registro de evidencias. Las seis evidencias E-01 a E-06 se entregan en el repositorio, dentro de docs/evidencias/, cada una en formato Markdown, con los comandos de verificacion, el resultado esperado y la parte que queda por ejecutar en la maquina del equipo. El indice completo y el procedimiento paso a paso para completar las capturas estan en docs/evidencias/README.md.

ID	Nombre de evidencia	Qué demuestra	Ubicación (informe/repositorio/ video)
E-01	Red virtual segmentada y reglas de seguridad	Que la solucion implementa dos capas de red con direccionamiento explicito, que la capa privada no tiene salida a Internet y que el acceso directo a la base de datos y al backend esta denegado.	Informe: seccion 5 (Figuras 1 y 2, tabla 6). Repositorio: docs/evidencias/E-01-red-y-seguridad.md
E-02	Computo: compilacion y arranque del backend	Que el backend compila y arranca, y que expone el endpoint de salud con el estado de la base de datos. Incluye la matriz de control de acceso por roles.	Informe: secciones 6.1 y 6.3 (tablas 7 y 8). Repositorio: docs/evidencias/E-02-computo.md
E-03	Almacenamiento persistente	Que los datos de negocio se guardan en un volumen nombrado, que el ciclo de escritura y lectura funciona y que los datos sobreviven al ciclo de vida del contenedor.	Informe: seccion 6.2 (tabla 8). Repositorio: docs/evidencias/E-03-persistencia.md
E-04	Contenerizacion y comunicacion entre servicios	Que los tres componentes tienen Dockerfile o imagen oficial, que la configuracion multi-contenedor es valida, que las variables de entorno no contienen secretos y que la comunicacion navegador-frontend-backend-base de datos esta definida.	Informe: seccion 7 (tabla 9). Repositorio: docs/evidencias/E-04-contenedores.md
E-05	Repositorio, documentacion y control de secretos	Que el avance esta versionado en un repositorio ordenado, con commits trazables de los integrantes, documentacion completa y sin credenciales en el arbol versionado ni en el historial.	Informe: seccion 8 (tabla 10). Repositorio: docs/evidencias/E-05-repositorio.md
E-06	Observabilidad: logs, comprobaciones de salud y metricas	Como se observa el estado de los servicios, que indicadores de rendimiento conviene vigilar y que criterio inicial justificaria escalar o ajustar recursos.	Informe: seccion 10 (tabla 12). Repositorio: docs/evidencias/E-06-observabilidad.md